Scala Laboratory Work 3

Scala Best Practices and Idioms

Andrew Galashin

Six tasks covering immutable recursion, Option and collections, type inference, guarded pattern matching and asynchronous Future composition. The solutions were compiled and executed using Scala 2.13.16 and Java 21. All checks passed.

Task 1 Immutable Fibonacci sequence

Replace the mutable Fibonacci example with tail-recursive immutable state. The public signature remains Seq[Int], and the first ten values match the assignment exactly. Each recursive call passes the remaining count, the previous two values and a reversed immutable list.

```
def fibs(count: Int): Seq[Int] = {
  require(count >= 0, "The number of terms must not be negative")
  @tailrec
  def loop(left: Int, previous: Int, current: Int,
           reversed: List[Int]): List[Int] = {
    if (left == 0) reversed.reverse
    else {
      val next = Math.addExact(previous, current)
      loop(left - 1, current, next, next :: reversed)
    }
  }
  loop(count, 0, 1, Nil)
}
```

The compiler verifies tail recursion through @tailrec. Prepending takes constant time, and one final reverse restores order. Runtime is O(n), with O(n) output storage and constant call-stack space. There are no vars or mutable collections in fibs. Zero terms produce an empty list; negative counts are rejected. Math.addExact detects Int overflow: 45 terms fit, ending at 1836311903, whereas term 46 is rejected.

Task 2 Expand the expression using flatMap and map

```
people.toSeq.flatMap { entries =>
  entries.flatMap { personOption =>
    personOption.toSeq.flatMap { person =>
      person.address.toSeq.map { address => address.zipCode }
    }
  }
}
```

Here people has type Option[Seq[Option[Person]]]. Each Option is converted to a zero-or-one-element Seq before the nested collection operations. The final map extracts the ZIP code; each outer flatMap removes one collection layer. For the supplied Fred, Simon and absent-person data, the result is List(12345). Missing people and addresses contribute no element.

Without the outer conversion, Option.flatMap expects a function returning Option[B], but entries.flatMap returns Seq[String]. Those types are incompatible. A deliberately broken expansion reproduces this exact compiler error; the recorded evidence is included below.

Task 3 Determine the inferred return type

```
def safeDiv(n: Int, d: Int) = if (d == 0) None else n / d
```

Under Scala 2.13, the inferred return type is Any. The first branch has type None.type and the second has type Int. Their common supertype is Any, not Option[Int]. Scala does not automatically wrap n / d in Some. The compiler typer output confirms:

```
def safeDiv(n: Int, d: Int): Any = ...
```

safeDivOriginal(5, 0) returns None, while safeDivOriginal(5, 2) returns the bare integer 2. The method accepts Int arguments and returns Any; as a function value, its type is (Int, Int) => Any.

Task 4 Explain flatten and fix the return type

The assignment repeats the same function in Task 4, so its original inferred type is again Any. Mapping it over List(0,1,2) gives List[Any]. flatten cannot turn arbitrary Any values into elements because they are not known to be iterable. Merely adding : Option[Int] also fails: the nonzero branch still returns Int.

```
def safeDiv(n: Int, d: Int): Option[Int] =
  if (d == 0) None else Some(n / d)

List(0, 1, 2).map(safeDiv(5, _))
// List(None, Some(5), Some(2))
List(0, 1, 2).map(safeDiv(5, _)).flatten
// List(5, 2)
List(0, 1, 2).flatMap(safeDiv(5, _))
// List(5, 2)
```

The corrected return type is Option[Int]. None contributes no value; Some contributes one value during flatten or flatMap. Division remains ordinary Scala Int division, truncating toward zero. Compiler evidence includes both the failing flatten and the invalid annotated version.

Task 5 Rewrite with a match and a guard

```
def describeSign(n: Int): String = n match {
  case 0 => "Zero"
  case value if value > 0 => "Positive"
  case _ => "Negative"
}
```

The first case handles zero. The guard on the second case accepts positive integers. The final wildcard handles all negative integers, making the match exhaustive. Tests include -5, 0, 7, Int.MinValue and Int.MaxValue. No negation or absolute-value operation is needed.

Task 6 Rewrite the computation asynchronously

The original computation adds 1, 2 and 3, then formats The answer is 6. Independent input futures are created before composition, so they are eligible to run concurrently. The computation returns Future[String] immediately.

```
val a = Future(1)
val b = Future(2)
val c = Future(3)
val text = Future("The answer is")
val result: Future[String] = answerAsync(a, b, c, text)
```

```
def answerAsync(a: Future[Int], b: Future[Int], c: Future[Int],
                text: Future[String])
               (implicit ec: ExecutionContext): Future[String] =
  for {
    av <- a
    bv <- b
    cv <- c
    prefix <- text
    sum <- Future(av + bv + cv)
  } yield s"$prefix $sum"
```

A for comprehension composes Futures using flatMap and map; it does not block a worker while waiting for another Future. The source also includes an equivalent answerWithFlatMap implementation. Both produce The answer is 6.

ExecutionContext is supplied implicitly by the caller. No Await, Thread.sleep or blocking operation occurs inside either asynchronous function. Await is used only at the terminal/test boundary to keep the application alive until the result is observed, with a five-second timeout.

A Promise test leaves the first input unresolved and confirms that the composition has already returned an incomplete Future. Completing that Promise yields the expected string. Another test supplies a failed Future and verifies that the same exception is propagated. These checks do not rely on timing delays.

Verification and reproduction

Lab3.scala contains all six solutions and 16 passing checks. The main source compiles with -deprecation -feature -Werror. The deliberately failing examples are kept in compiler-examples and compiled separately. Their actual compiler output is in evidence.

```
scalac -deprecation -feature -Werror Lab3.scala
scala Lab3
```

In the prepared local folder, Run.cmd rebuilds and executes the application. Scala compiler JARs are kept under tools on D. The submission archive excludes tools, class files and rendering previews.

References

Assignment: Laboratory Work N3, Scala Best Practices and Idioms, tasks on PDF pages 3, 6, 15, 16, 17 and 28.

Scala Option API: https://www.scala-lang.org/api/2.13.16/scala/Option.html

Scala Futures and Promises: https://docs.scala-lang.org/overviews/core/futures.html

Appendix A Actual execution output

```
SCALA LABORATORY WORK 3 - BEST PRACTICES AND IDIOMS
Author: Andrew Galashin
Scala 2.13.16; Java 21.0.13.0.2
TASK 1 - Immutable tail-recursive Fibonacci
fibs(10) = List(1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
PASS: supplied ten-term sequence
PASS: zero, one and two terms
PASS: negative term count rejected
PASS: Int boundary and overflow detection
PASS: length and recurrence for all supported counts 0 to 45
TASK 2 - Options and collections using flatMap and map
zipCodes = List(12345)
PASS: explicit expression equals for comprehension
PASS: absent outer option, empty sequence and missing values
PASS: multiple addresses preserve order and duplicates
TASK 3 - Inferred return type of the original safeDiv
Scala 2.13 compiler inference: Any (see evidence/Type-inference.txt)
safeDivOriginal(5, 0) = None
safeDivOriginal(5, 2) = 2
PASS: original returns None or a bare Int, not Option[Int]
TASK 4 - Correct Option[Int] return type and flatten
map = List(None, Some(5), Some(2))
flatten = List(5, 2)
PASS: safe division and flatMap
PASS: negative operands and zero numerator
TASK 5 - Pattern matching with a guard
describeSign(-5) = Negative
describeSign(0) = Zero
describeSign(7) = Positive
PASS: negative, zero, positive and Int extremes
TASK 6 - Fully asynchronous composition
Future result = The answer is 6
PASS: for and flatMap versions produce The answer is 6
PASS: composition returns before an unresolved input is completed
PASS: pending input eventually yields the correct result
PASS: a failed input is propagated as the same exception
ALL CHECKS PASSED
```

Appendix B Compiler evidence

These are actual Scala 2.13.16 diagnostics. The broken examples are intentionally excluded from the successful application.

Inferred return type

```
[[syntax trees at end of                     typer]] // TypeProbe.scala
package <empty> {
  object TypeProbe extends scala.AnyRef {
    def <init>(): TypeProbe.type = {
      TypeProbe.super.<init>();
      ()
    };
    def safeDiv(n: Int, d: Int): Any = if (d.==(0))
      scala.None
    else
      n./(d)
  }
}
```

BrokenOptionMix

```
D:\HomeWork\Functional programming\H3\compiler-examples\BrokenOptionMix.scala:3: error: type mismatch;
 found   : Seq[String]
 required: Option[?]
  val result = people.flatMap(entries => entries.flatMap(person => person.map(identity)))
                                                                  ^
1 error
```

BrokenFlatten

```
D:\HomeWork\Functional programming\H3\compiler-examples\BrokenFlatten.scala:3: error: No implicit view
    available from Any => scala.collection.IterableOnce[B].
  val result = List(0, 1, 2).map(safeDiv(5, _)).flatten
                                                 ^
1 error
```

BrokenOptionReturn

```
D:\HomeWork\Functional programming\H3\compiler-examples\BrokenOptionReturn.scala:2: error: type mismatch;
 found   : Int
 required: Option[Int]
  def safeDiv(n: Int, d: Int): Option[Int] = if (d == 0) None else n / d
                                                                    ^
1 error
```

Appendix C Complete Scala source

```
import scala.annotation.tailrec
import scala.concurrent.{Await, ExecutionContext, Future, Promise}
import scala.concurrent.duration._
import scala.util.{Failure, Success, Try}
object Lab3 {
  // Task 1: preserve the assignment's Seq[Int] signature and 1,2,3,... order.
  // An overflowing Int is rejected instead of silently wrapping around.
  def fibs(count: Int): Seq[Int] = {
    require(count >= 0, "The number of terms must not be negative")
    @tailrec
    def loop(left: Int, previous: Int, current: Int,
             reversed: List[Int]): List[Int] = {
      if (left == 0) reversed.reverse
      else {
        val next = Math.addExact(previous, current)
        loop(left - 1, current, next, next :: reversed)
      }
    }
    loop(count, 0, 1, Nil)
  }
  case class Address(street: String, city: String, state: String, zipCode: String)
  case class Person(first: String, last: String, address: Option[Address])
  // Task 2: explicit map/flatMap equivalent, with no for comprehension.
  def zipCodes(people: Option[Seq[Option[Person]]]): Seq[String] =
    people.toSeq.flatMap { entries =>
      entries.flatMap { personOption =>
        personOption.toSeq.flatMap { person =>
          person.address.toSeq.map { address => address.zipCode }
        }
      }
    }
  def zipCodesFor(people: Option[Seq[Option[Person]]]): Seq[String] =
    for {
      entries <- people.toSeq
      personOption <- entries
      person <- personOption.toSeq
      address <- person.address.toSeq
    } yield address.zipCode
  // Task 3: intentionally leave the return type inferred, exactly as assigned.
  def safeDivOriginal(n: Int, d: Int) = if (d == 0) None else n / d
  // Task 4: both branches now have the common, useful type Option[Int].
  // Division truncates toward zero, following Scala Int division.
  def safeDiv(n: Int, d: Int): Option[Int] =
    if (d == 0) None else Some(n / d)
  // Task 5: match with a guard; no negation or abs that could overflow.
  def describeSign(n: Int): String = n match {
    case 0 => "Zero"
    case value if value > 0 => "Positive"
    case _ => "Negative"
  }
  // Task 6: no Await, blocking, or sleep inside these asynchronous functions.
  def answerAsync(a: Future[Int], b: Future[Int], c: Future[Int],
                  text: Future[String])(implicit ec: ExecutionContext): Future[String] =
    for {
      av <- a
      bv <- b
      cv <- c
      prefix <- text
      sum <- Future(av + bv + cv)
    } yield s"$prefix $sum"
  def answerWithFlatMap(a: Future[Int], b: Future[Int], c: Future[Int],
                        text: Future[String])
                       (implicit ec: ExecutionContext): Future[String] =
    a.flatMap(av => b.flatMap(bv => c.flatMap(cv =>
      text.flatMap(prefix => Future(av + bv + cv).map(sum => s"$prefix $sum")))))
  def check(label: String)(condition: => Boolean): Unit = {
    assert(condition, label)
    println(s"PASS: $label")
  }
  def main(args: Array[String]): Unit = {
    implicit val ec: ExecutionContext = ExecutionContext.global
```

```
println("SCALA LABORATORY WORK 3 - BEST PRACTICES AND IDIOMS")
println("Author: Andrew Galashin")
println(s"Scala ${scala.util.Properties.versionNumberString}; Java
${System.getProperty("java.version")}")
println("TASK 1 - Immutable tail-recursive Fibonacci")
println(s"fibs(10) = ${fibs(10)}")
check("supplied ten-term sequence") {
  fibs(10) == List(1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
}
check("zero, one and two terms") {
  fibs(0).isEmpty && fibs(1) == List(1) && fibs(2) == List(1, 2)
}
check("negative term count rejected") {
  Try(fibs(-1)).failed.toOption.exists(_.isInstanceOf[IllegalArgumentException])
}
check("Int boundary and overflow detection") {
  fibs(45).last == 1836311903 &&
    Try(fibs(46)).failed.toOption.exists(_.isInstanceOf[ArithmeticException])
}
check("length and recurrence for all supported counts 0 to 45") {
  (0 to 45).forall { n =>
    val xs = fibs(n)
    xs.length == n && xs.sliding(3).filter(_.length == 3).forall(t => t(2) == t(0) + t(1))
  }
}
println("TASK 2 - Options and collections using flatMap and map")
val p1 = Some(Person("Fred", "Bloggs", None))
val p2 = Some(Person("Simon", "Jones",
  Some(Address("123 Main", "Fakesville", "AZ", "12345"))))
val p3: Option[Person] = None
val people: Option[Seq[Option[Person]]] = Some(Seq(p1, p2, p3))
println(s"zipCodes = ${zipCodes(people)}")
check("explicit expression equals for comprehension") {
  zipCodes(people) == Seq("12345") && zipCodes(people) == zipCodesFor(people)
}
check("absent outer option, empty sequence and missing values") {
  zipCodes(None).isEmpty && zipCodes(Some(Seq.empty)).isEmpty &&
    zipCodes(Some(Seq(p1, p3))).isEmpty
}
check("multiple addresses preserve order and duplicates") {
  zipCodes(Some(Seq(p2, p2))) == Seq("12345", "12345")
}
println("TASK 3 - Inferred return type of the original safeDiv")
println("Scala 2.13 compiler inference: Any (see evidence/Type-inference.txt)")
println(s"safeDivOriginal(5, 0) = ${safeDivOriginal(5, 0)}")
println(s"safeDivOriginal(5, 2) = ${safeDivOriginal(5, 2)}")
check("original returns None or a bare Int, not Option[Int]") {
  safeDivOriginal(5, 0) == None && safeDivOriginal(5, 2) == 2
}
println("TASK 4 - Correct Option[Int] return type and flatten")
val mapped = List(0, 1, 2).map(safeDiv(5, _))
println(s"map = $mapped")
println(s"flatten = ${mapped.flatten}")
check("safe division and flatMap") {
  mapped == List(None, Some(5), Some(2)) && mapped.flatten == List(5, 2) &&
    List(0, 1, 2).flatMap(safeDiv(5, _)) == List(5, 2)
}
check("negative operands and zero numerator") {
  safeDiv(-5, 2).contains(-2) && safeDiv(5, -2).contains(-2) && safeDiv(0, 7).contains(0)
}
println("TASK 5 - Pattern matching with a guard")
List(-5, 0, 7).foreach(n => println(s"describeSign($n) = ${describeSign(n)}"))
check("negative, zero, positive and Int extremes") {
  describeSign(-5) == "Negative" && describeSign(0) == "Zero" &&
    describeSign(7) == "Positive" && describeSign(Int.MinValue) == "Negative" &&
    describeSign(Int.MaxValue) == "Positive"
}
println("TASK 6 - Fully asynchronous composition")
// Independent futures are created before composition, so they can run concurrently.
val a = Future(1)
val b = Future(2)
val c = Future(3)
val text = Future("The answer is")
val result = answerAsync(a, b, c, text)
```

```
    // Only the terminal/test boundary waits, keeping the JVM alive for results.
    val value = Await.result(result, 5.seconds)
    println(s"Future result = $value")
    check("for and flatMap versions produce The answer is 6") {
      value == "The answer is 6" &&
        Await.result(answerWithFlatMap(a, b, c, text), 5.seconds) == value
    }
    val pending = Promise[Int]()
    val deferred = answerAsync(pending.future, b, c, text)
    check("composition returns before an unresolved input is completed") {
      !deferred.isCompleted
    }
    pending.success(1)
    check("pending input eventually yields the correct result") {
      Await.result(deferred, 5.seconds) == "The answer is 6"
    }
    val failure = new IllegalStateException("input failed")
    val failed = answerAsync(Future.failed(failure), b, c, text)
    check("a failed input is propagated as the same exception") {
      Await.result(failed.failed, 5.seconds) eq failure
    }
    println("ALL CHECKS PASSED")
  }
}
```